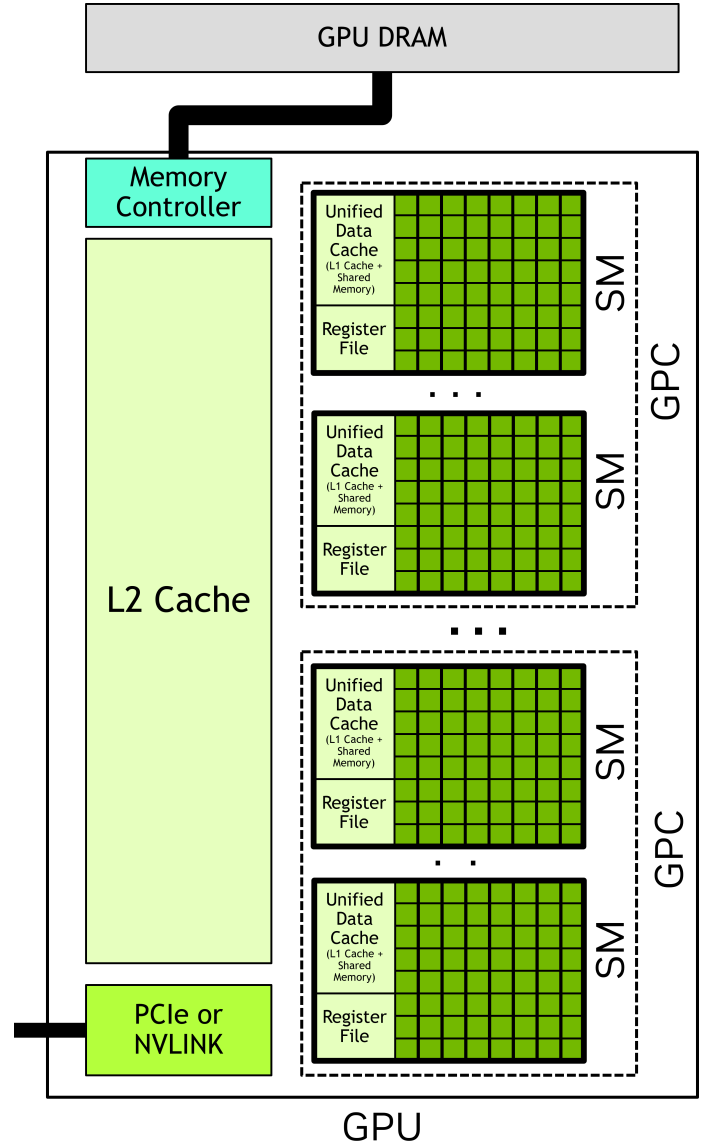
Matrix Multiplication on GPUs

Max Koch

GPU Architecture Recap

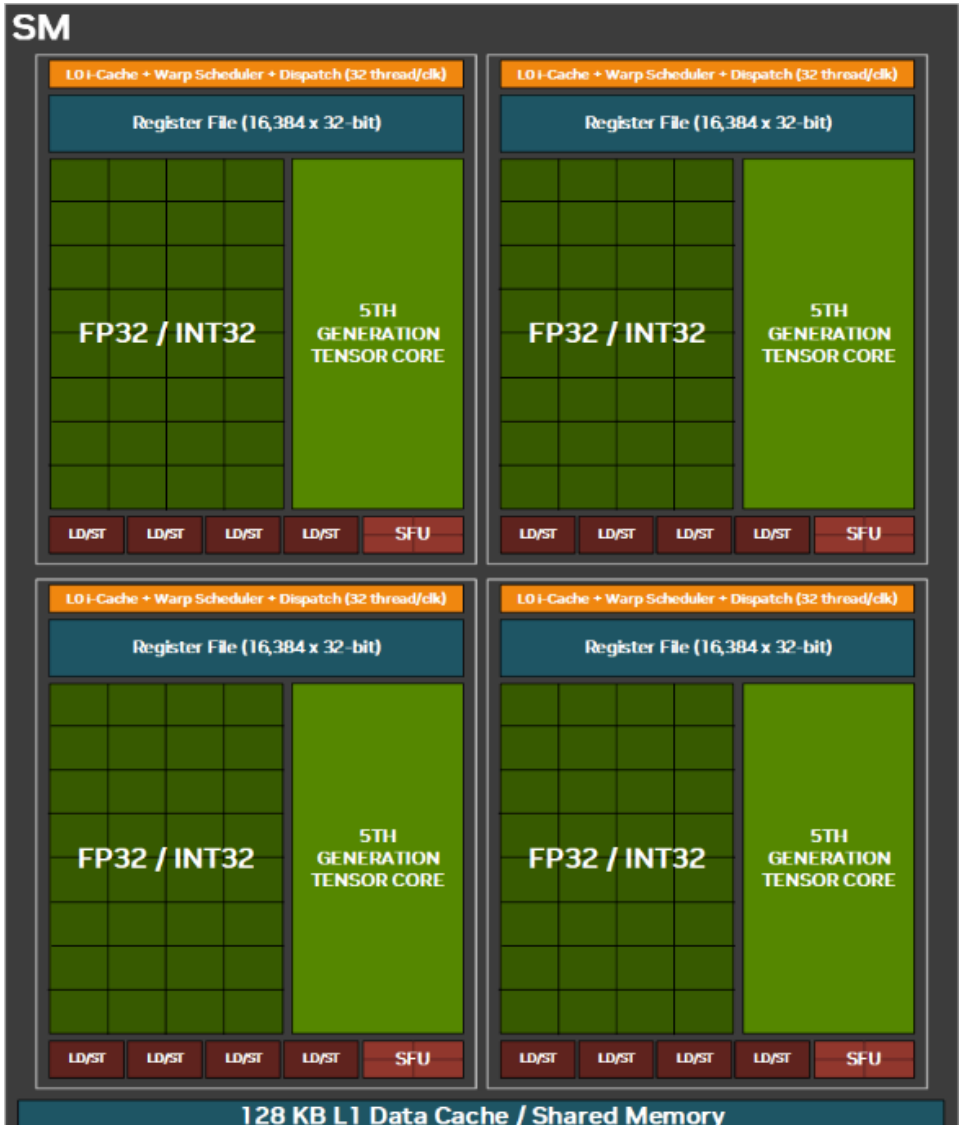
- GPUs have an array of **Streaming Multiprocessors (SMs)**
- Each SM is an independent parallel processor
- All SMs share an **L2 cache**
- Off-chip **HBM** (High Bandwidth Memory) — high bandwidth, high latency
- CPU and GPU communicate over PCIe or NVLink



GPU system with SM array, shared L2, and HBM
Adapted from: NVIDIA — CUDA Programming Guide

GPU Architecture Recap

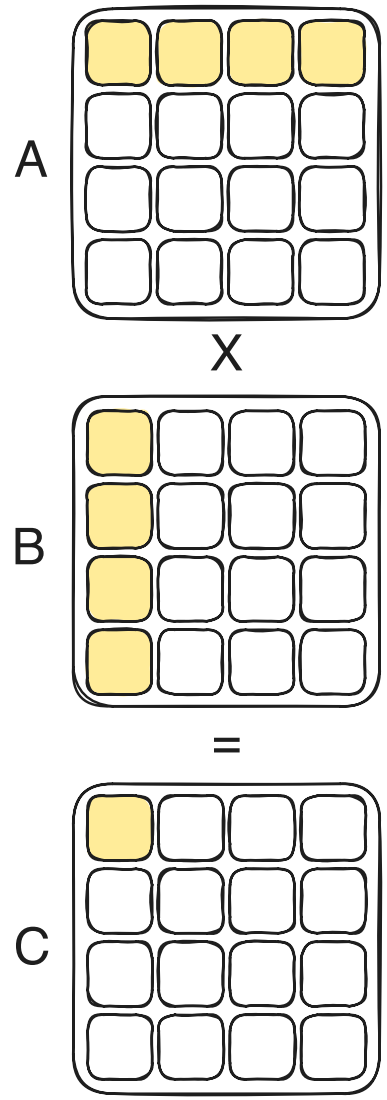
- Each SM contains:
  - **CUDA cores**: contain ALUs for scalar integer and floating-point ops
  - **Tensor Cores**: hardware matrix multiply-accumulate (MMA)
  - **Shared memory / L1 cache**: fast on-chip scratchpad memory
  - **Register file**: large, per-thread
  - **Warp schedulers**: dispatch groups of 32 threads, fast context switching
- Warps execute in **SIMT**: single instruction, multiple threads



The Blackwell Streaming Multiprocessor (SM)
Adapted from: NVIDIA — NVIDIA RTX Blackwell GPU Architecture

Matrix Multiplication

- One of the most important kernels in deep learning
- Used for:
  - Model training and inference
  - Rendering graphics
  - Scientific computing
  - …



$$C = A \times B$$

Matrix Multiplication with CUDA Cores

- Each thread is responsible for computing one (or multiple) output elements of C
- For each output element, the thread loops over the reduction dimension and accumulates the products of corresponding elements from A and B

- **Problems:**
  - Different threads have to load the same elements of A and B to their respective registers
  - Each thread has to load, compute, and store elements in each iteration of the reduction loop
  - Support for *mixed precision* is limited (would require manual dtype promotion)

Tensor Cores solve these problems.

Tensor Cores

- Specialized hardware units introduced with **Volta** (2017)
- Execute matrix multiply-accumulate $D = A \times B + C$ in a single instruction
- Operate on **fixed tile shapes**
- Support **mixed precision** with automatic accumulation in higher precision
- Programming perspective: Multiple threads collaborate on one matrix multiplication

Blackwell Tensor Core — FP4
From: NVIDIA — NVIDIA RTX Blackwell GPU Architecture

Tensor Cores: Element Assignment to Registers (Volta)

- Each thread is responsible for loading a specific subset of elements of A and B into registers

	0	1	2	3	4	5	6	7
0	T0 V0	T0 V1	T0 V2	T0 V3	T16 V0	T16 V1	T16 V2	T16 V3
1	T1 V0	T1 V1	T1 V2	T1 V3	T17 V0	T17 V1	T17 V2	T17 V3
2	T2 V0	T2 V1	T2 V2	T2 V3	T18 V0	T18 V1	T18 V2	T18 V3
3	T3 V0	T3 V1	T3 V2	T3 V3	T19 V0	T19 V1	T19 V2	T19 V3

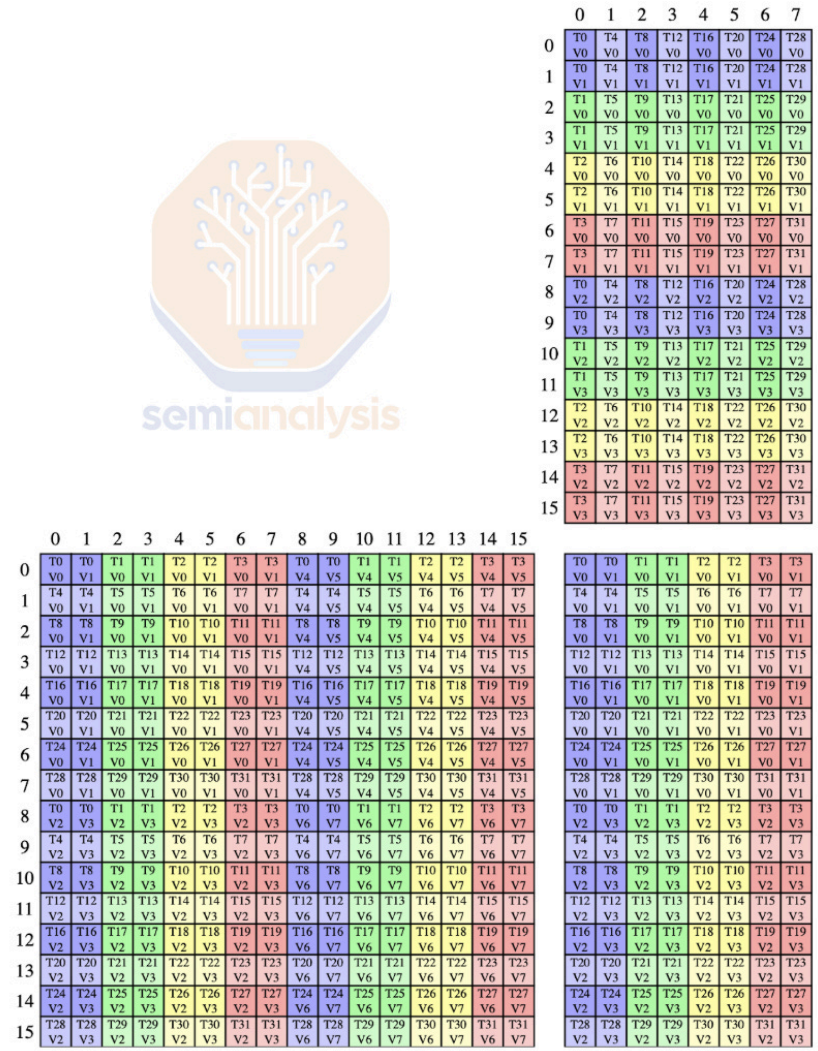
	0	1	2	3
0	T0 V0	T1 V0	T2 V0	T3 V0
1	T0 V1	T1 V1	T2 V1	T3 V1
2	T0 V2	T1 V2	T2 V2	T3 V2
3	T0 V3	T1 V3	T2 V3	T3 V3
4	T16 V0	T17 V0	T18 V0	T19 V0
5	T16 V1	T17 V1	T18 V1	T19 V1
6	T16 V2	T17 V2	T18 V2	T19 V2
7	T16 V3	T17 V3	T18 V3	T19 V3

T0 V0	T0 V1	T2 V0	T2 V1	T0 V4	T0 V5	T2 V4	T2 V5
T1 V0	T1 V1	T3 V0	T3 V1	T1 V4	T1 V5	T3 V4	T3 V5
T0 V2	T0 V3	T2 V2	T2 V3	T0 V6	T0 V7	T2 V6	T2 V7
T1 V2	T1 V3	T3 V2	T3 V3	T1 V6	T1 V7	T3 V6	T3 V7
T16 V0	T16 V1	T18 V0	T18 V1	T16 V4	T16 V5	T18 V4	T18 V5
T17 V0	T17 V1	T19 V0	T19 V1	T17 V4	T17 V5	T19 V4	T19 V5
T16 V2	T16 V3	T18 V2	T18 V3	T16 V6	T16 V7	T18 V6	T18 V7
T17 V2	T17 V3	T19 V2	T19 V3	T17 V6	T17 V7	T19 V6	T19 V7

Register layout on Volta
From: SemiAnalysis — NVIDIA Tensor Core Evolution: From Volta To Blackwell

Tensor Cores: Ampere

- Tensor Cores operate on larger tiles compared to Volta
- Thread assignments differ between Volta and Ampere
- Ampere's Tensor Cores operate at the full-warp level



Register layout on Ampere
From: SemiAnalysis — NVIDIA Tensor Core Evolution: From Volta To Blackwell

Tensor Cores: Matrix Sizes

- Different architectures support different tile sizes for A, B, and C

Tensor Core Size Increases			
Architecture	**Tensor Core FLOP/Cycle/SM**	**Max MMA Shape**	**Max FLOPs per PTX Instruction (2 * m * n * k)**
Volta	F16: 1024	m8n8k4	512
Ampere	F16: 2048	m16n8k16	4096
Hopper	F16: 4096 F8: 8192	**Warpgroup Level** **F16:** m64n256k16 **F8:** m64n256k64	**Warpgroup Level** **F16:** 524,288 **F8:** 2,097,152
Blackwell	F16: 8192 F8: 16384 F4: 32768	**2 SM** **F16:** m256n256k16 **F8:** m256n256k32 **F4:** m256n256k96	**2 SM** **F16:** 2,097,152 **F8:** 4,191,304 **F4:** 12,582,912

From: SemiAnalysis — NVIDIA Tensor Core Evolution: From Volta To Blackwell

Tensor Cores: Memory Levels

- Different architectures expect matrix elements to reside in different memory locations

Matrix Memory Residency across NVIDIA GPU Architectures			
Architecture	Matrix A	Matrix B	Matrix D
Volta	RF	RF	RF
Ampere	RF	RF	RF
Hopper	RF / SMEM	SMEM	RF
Blackwell	SMEM / TMEM	SMEM	TMEM

From: SemiAnalysis — NVIDIA Tensor Core Evolution: From Volta To Blackwell

Using Tensor Cores in cuTile

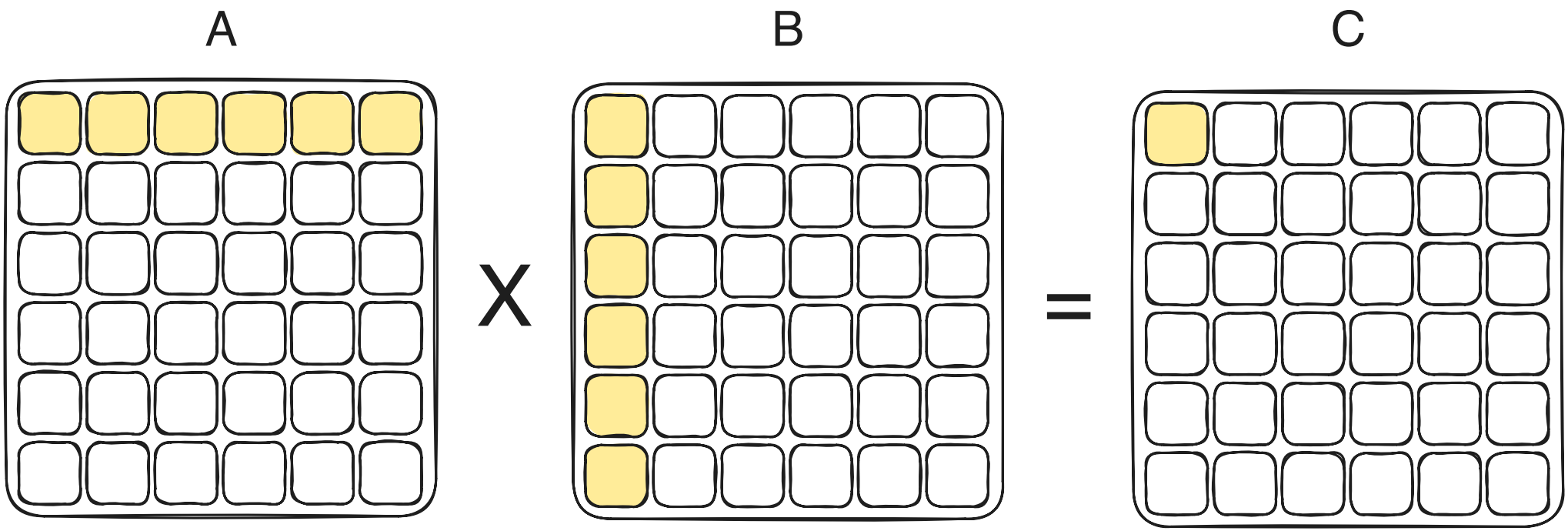
- cuTile provides a high-level API for programming Tensor Cores: `ct.mma`

```python
import cuda.tile as ct
import torch

@ct.kernel
def kernel():
    x = ct.ones((2, 4), dtype=ct.float16)
    y = ct.ones((4, 2), dtype=ct.float16)
    acc = ct.full((2, 2), 10.0, dtype=ct.float32)
    # (x @ y) + acc: each element = 1*4 + 10 = 14
    print(f"{ct.mma(x, y, acc):.1f}")


torch.cuda.init()
ct.launch(torch.cuda.current_stream(), (1,), kernel, ())
torch.cuda.synchronize()
```

Adapted from: NVIDIA — cuTile Python `cuda.tile.mma`

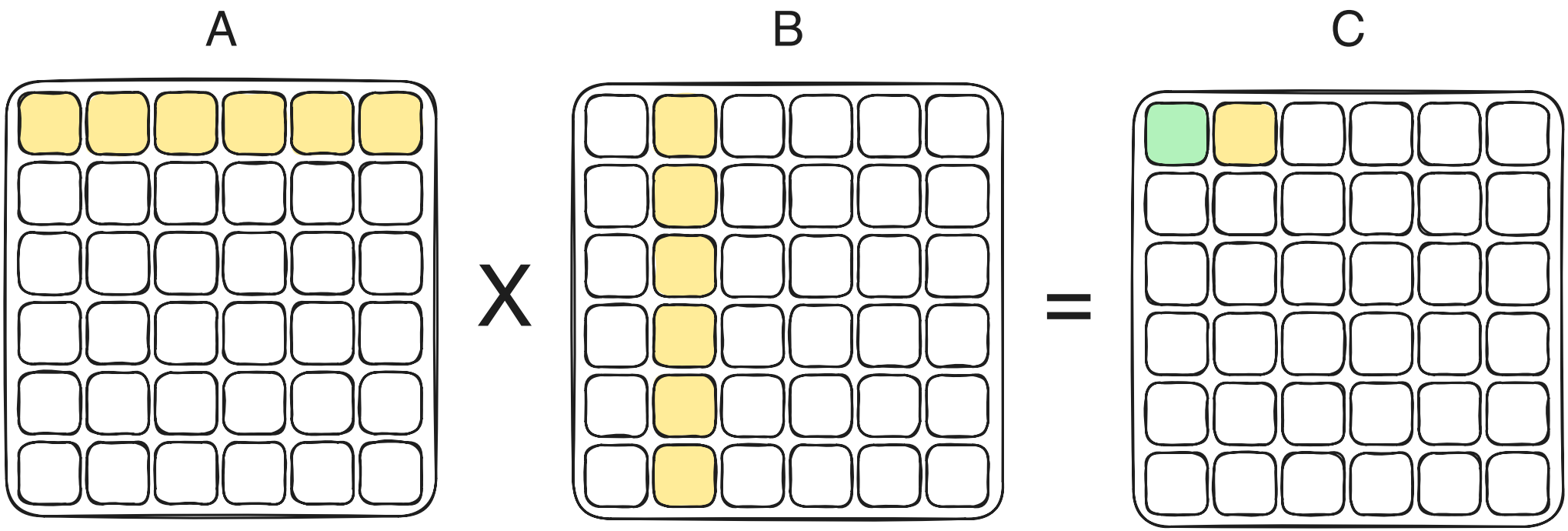
Tiled Matrix Multiplication

- For each output block of C, a block row of A has to be multiplied with a block column of B



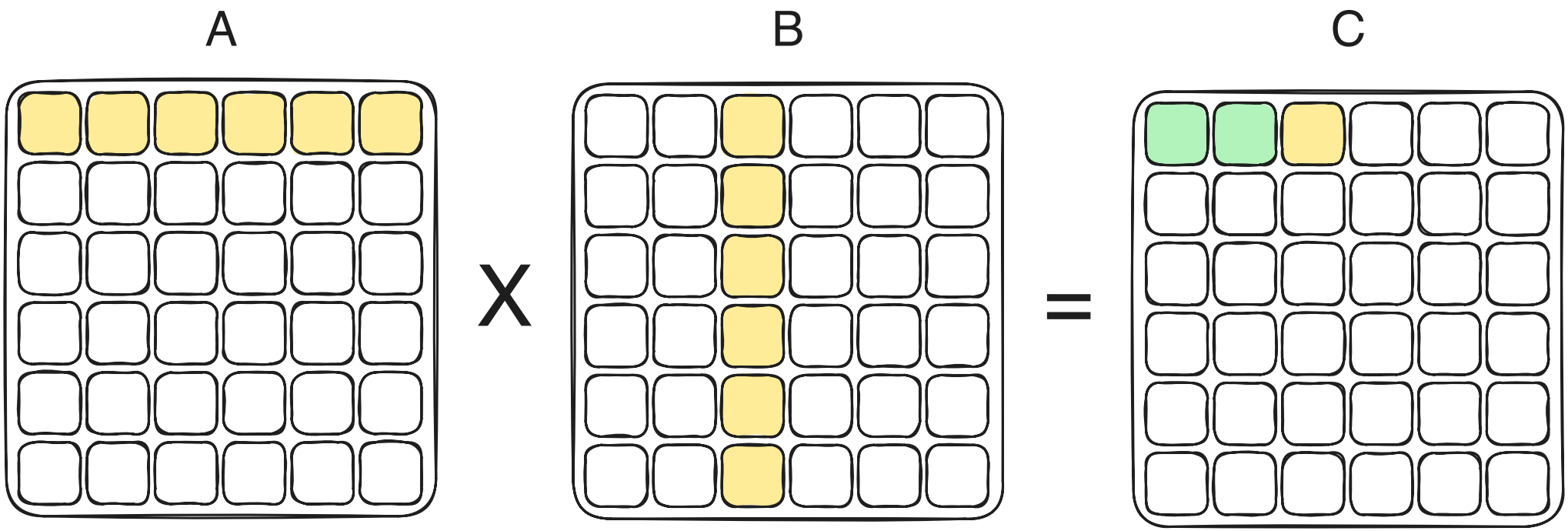
Tiled Matrix Multiplication

- For each output block of C, a block row of A has to be multiplied with a block column of B



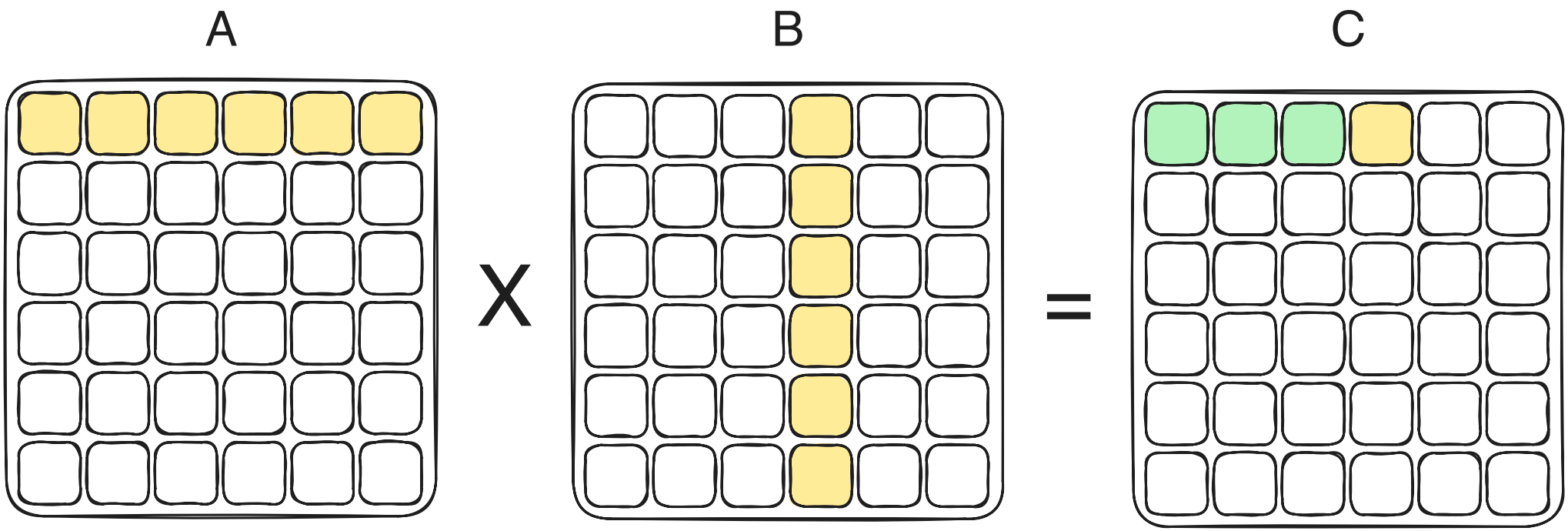
Tiled Matrix Multiplication

- For each output block of C, a block row of A has to be multiplied with a block column of B



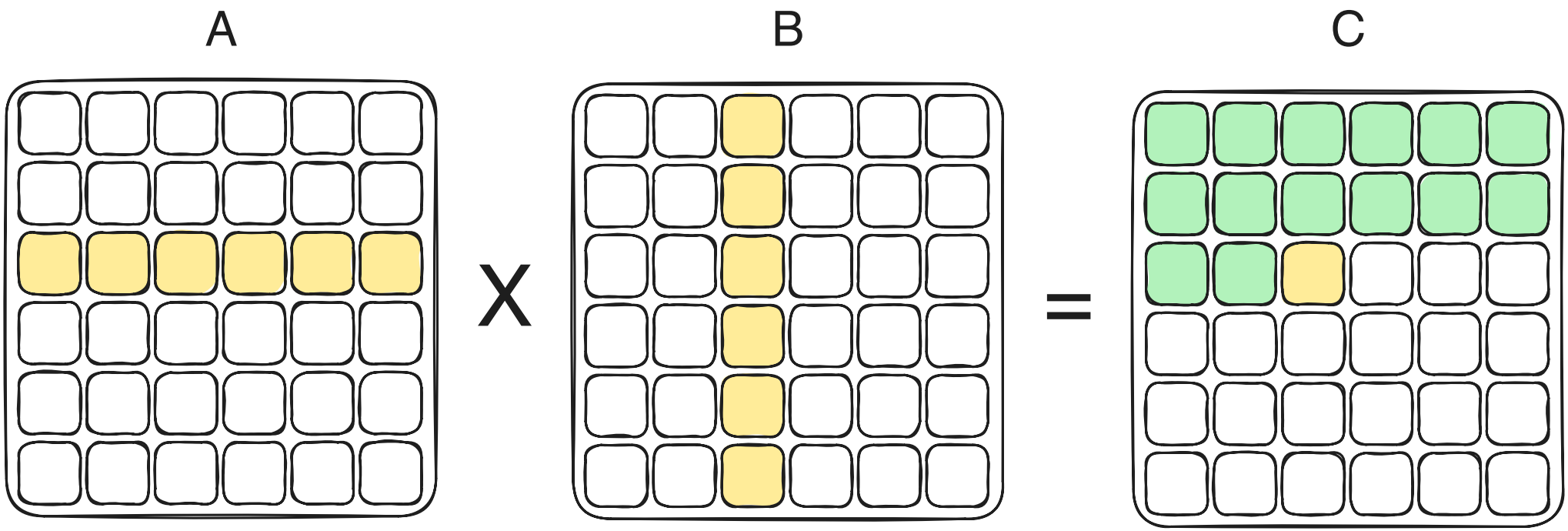
Tiled Matrix Multiplication

- For each output block of C, a block row of A has to be multiplied with a block column of B



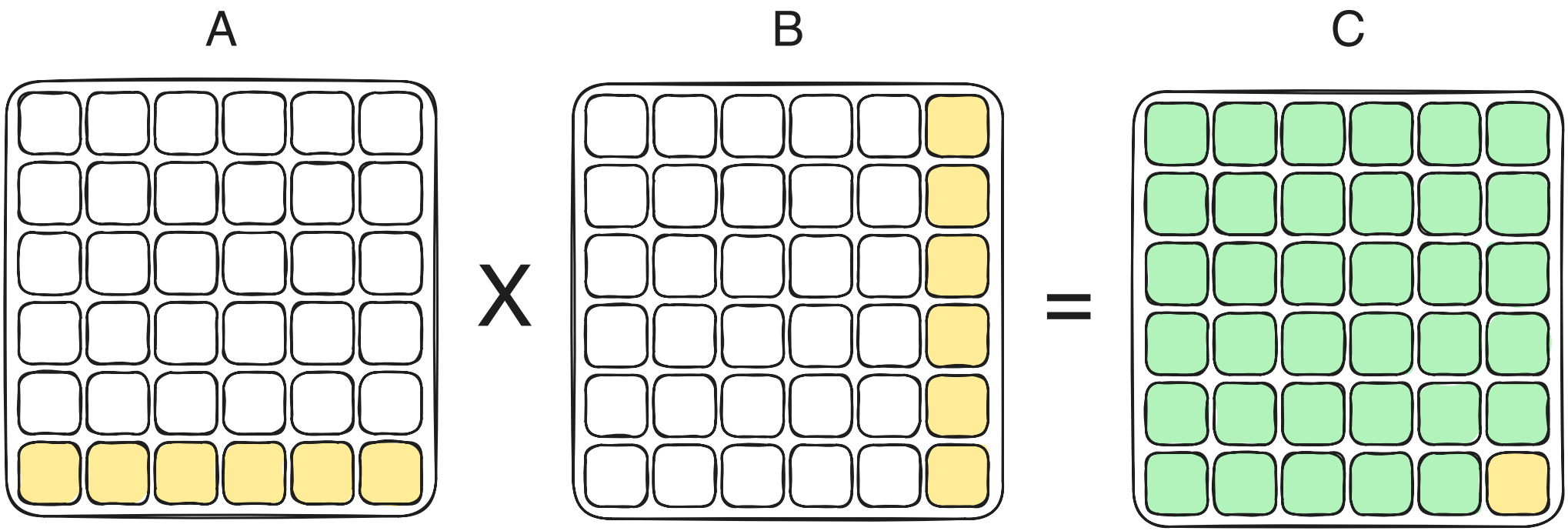
Tiled Matrix Multiplication

- For each output block of C, a block row of A has to be multiplied with a block column of B



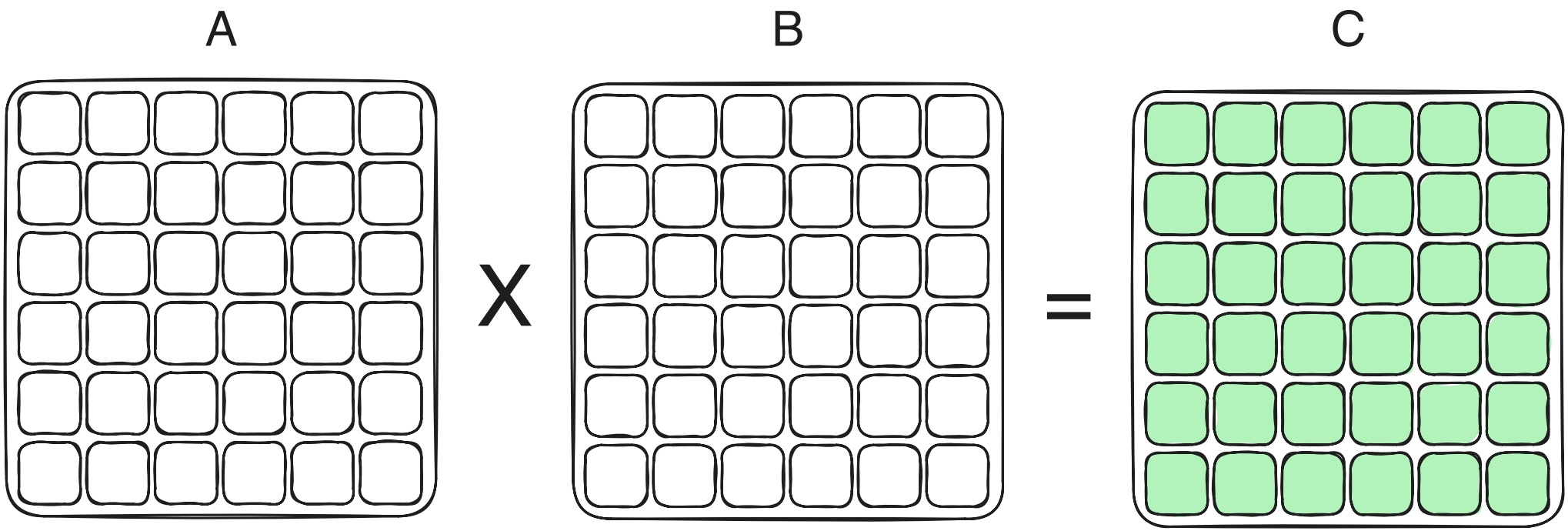
Tiled Matrix Multiplication

- For each output block of C, a block row of A has to be multiplied with a block column of B



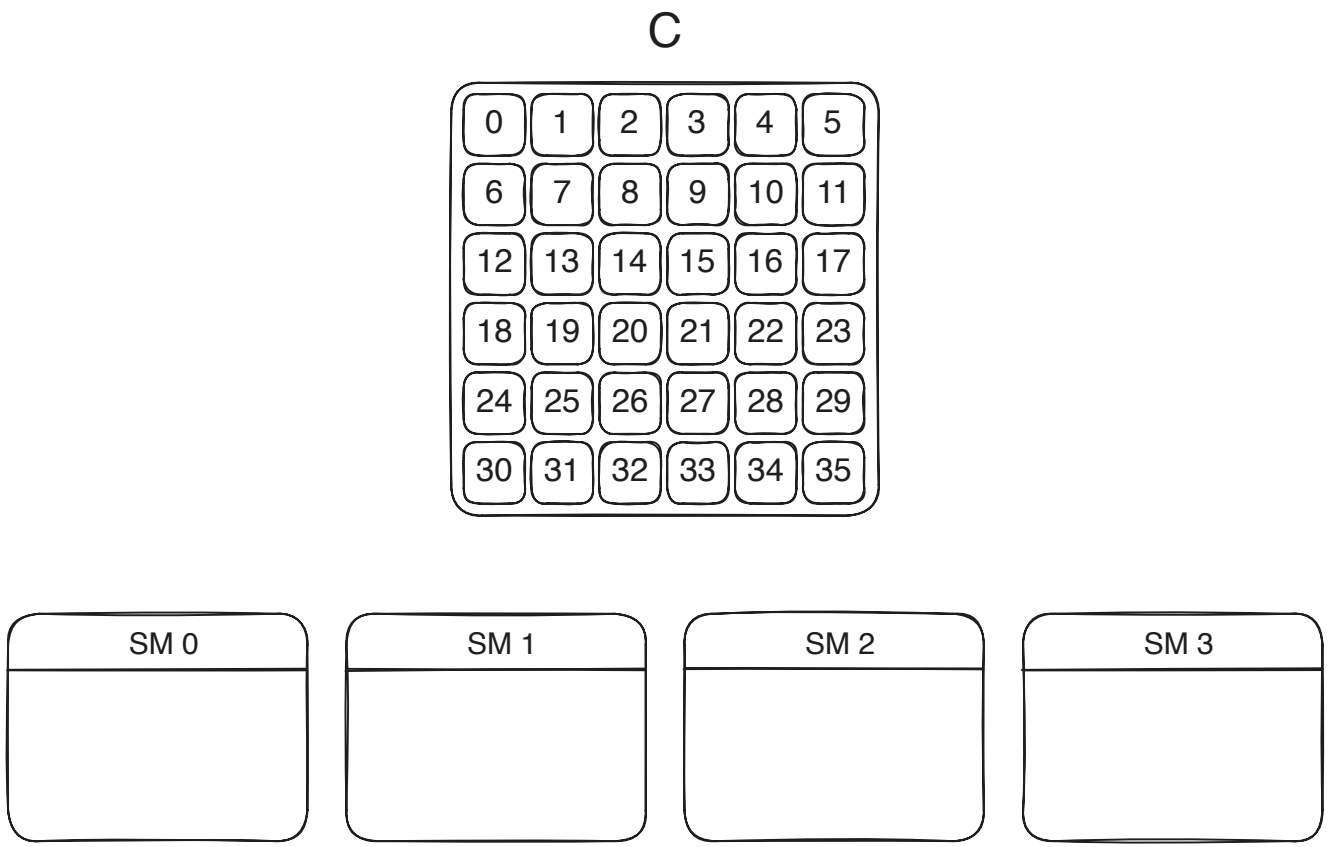
Tiled Matrix Multiplication

- For each output block of C, a block row of A has to be multiplied with a block column of B



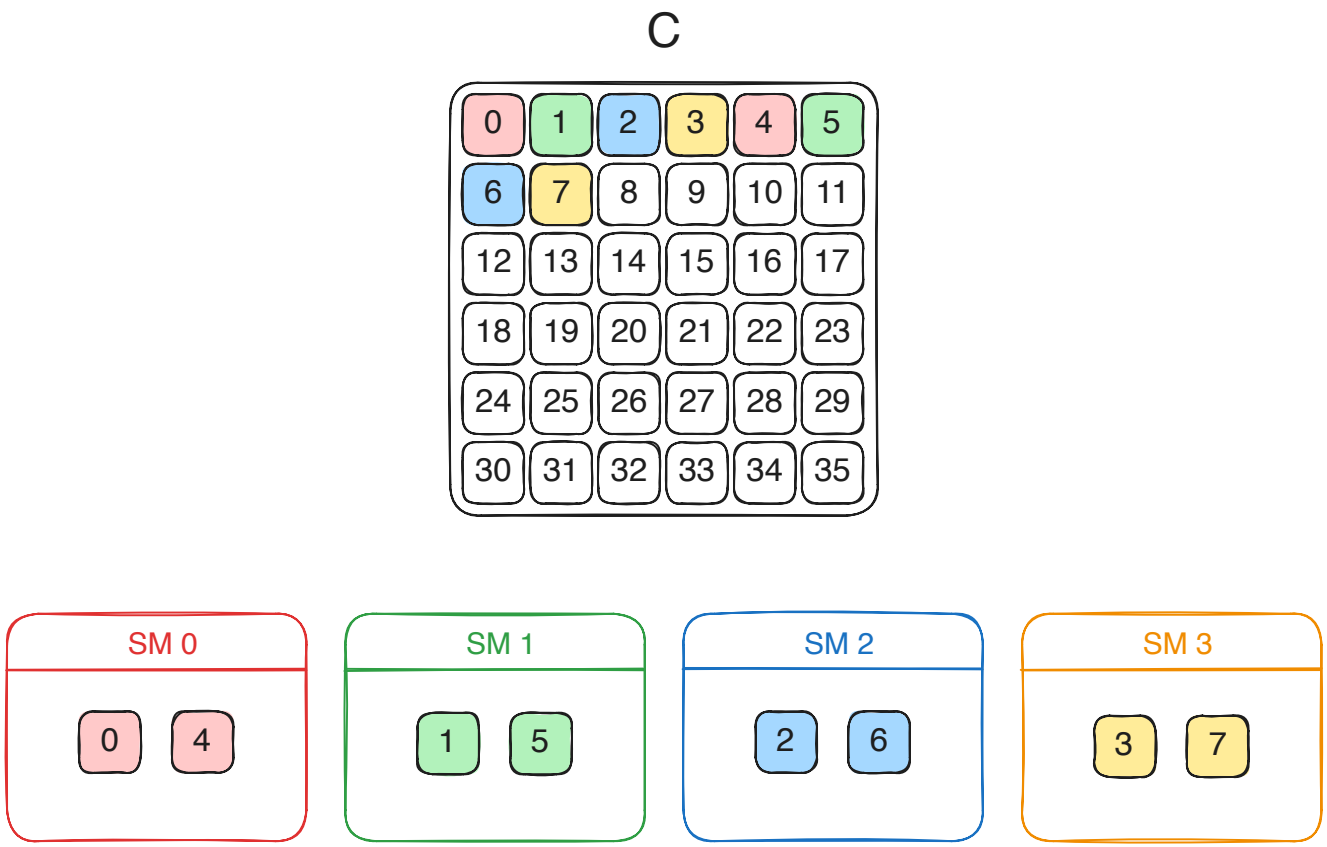
Assignment of Blocks to SMs

- Each output tile of matrix C can be computed in parallel



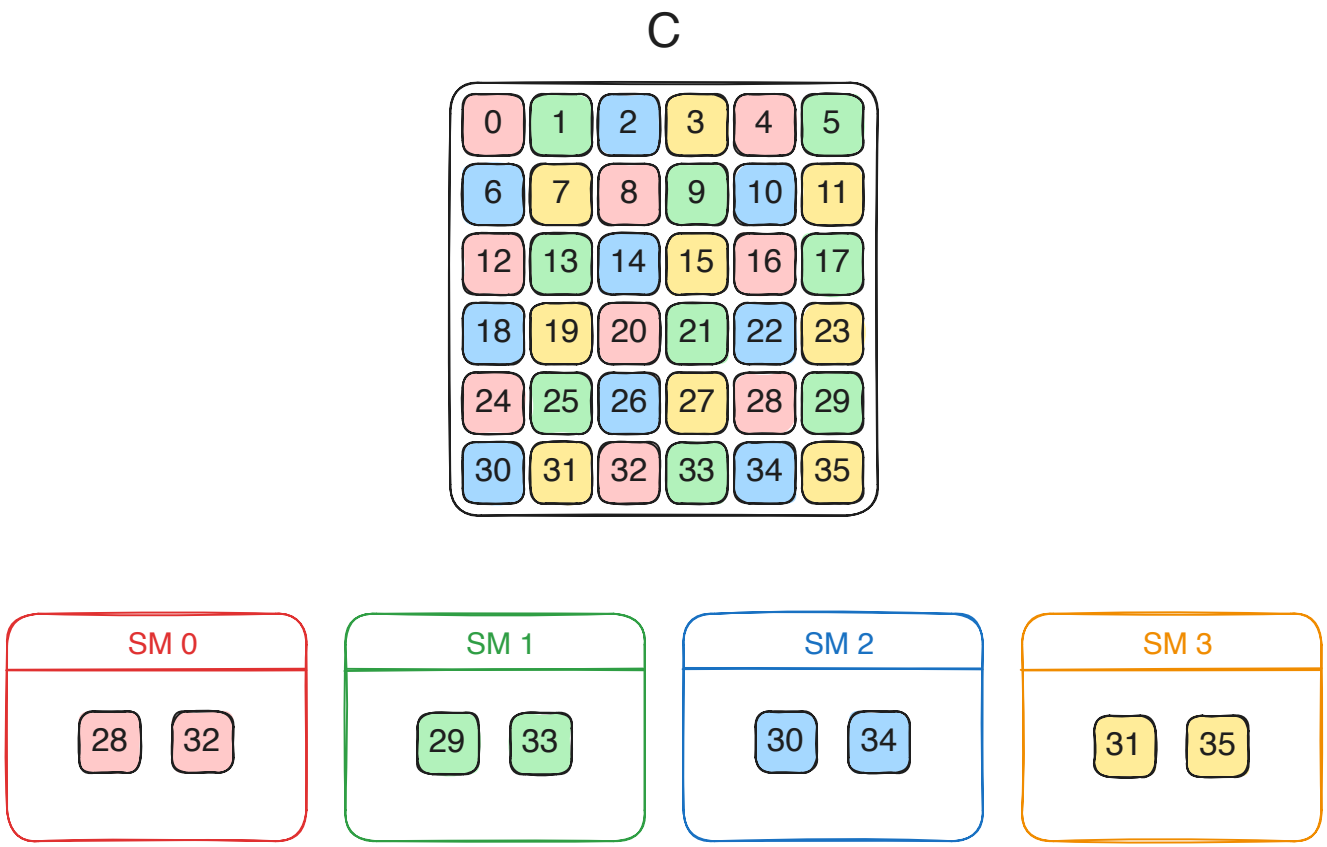
Assignment of Blocks to SMs

- Each output tile of matrix C can be computed in parallel



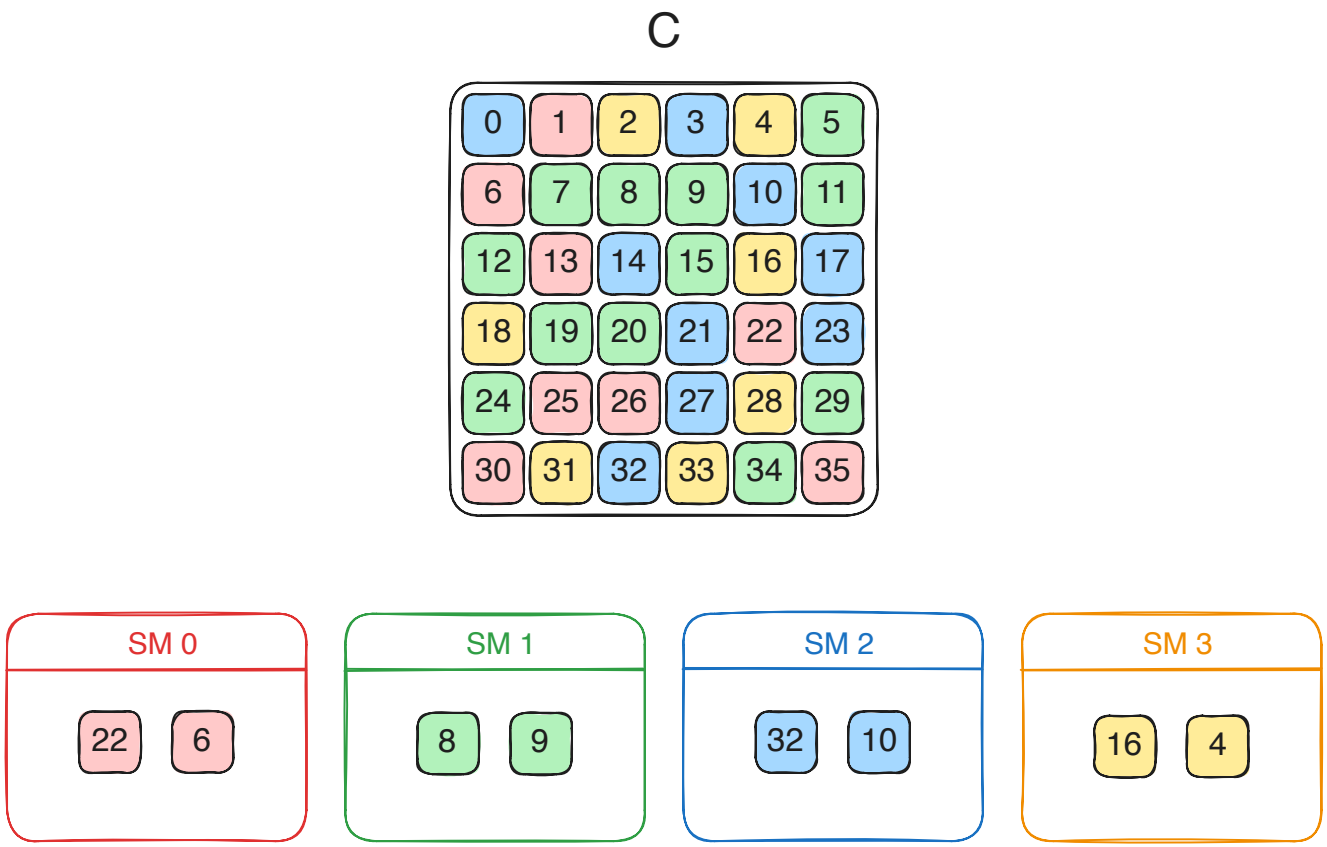
Assignment of Blocks to SMs

- Each output tile of matrix C can be computed in parallel



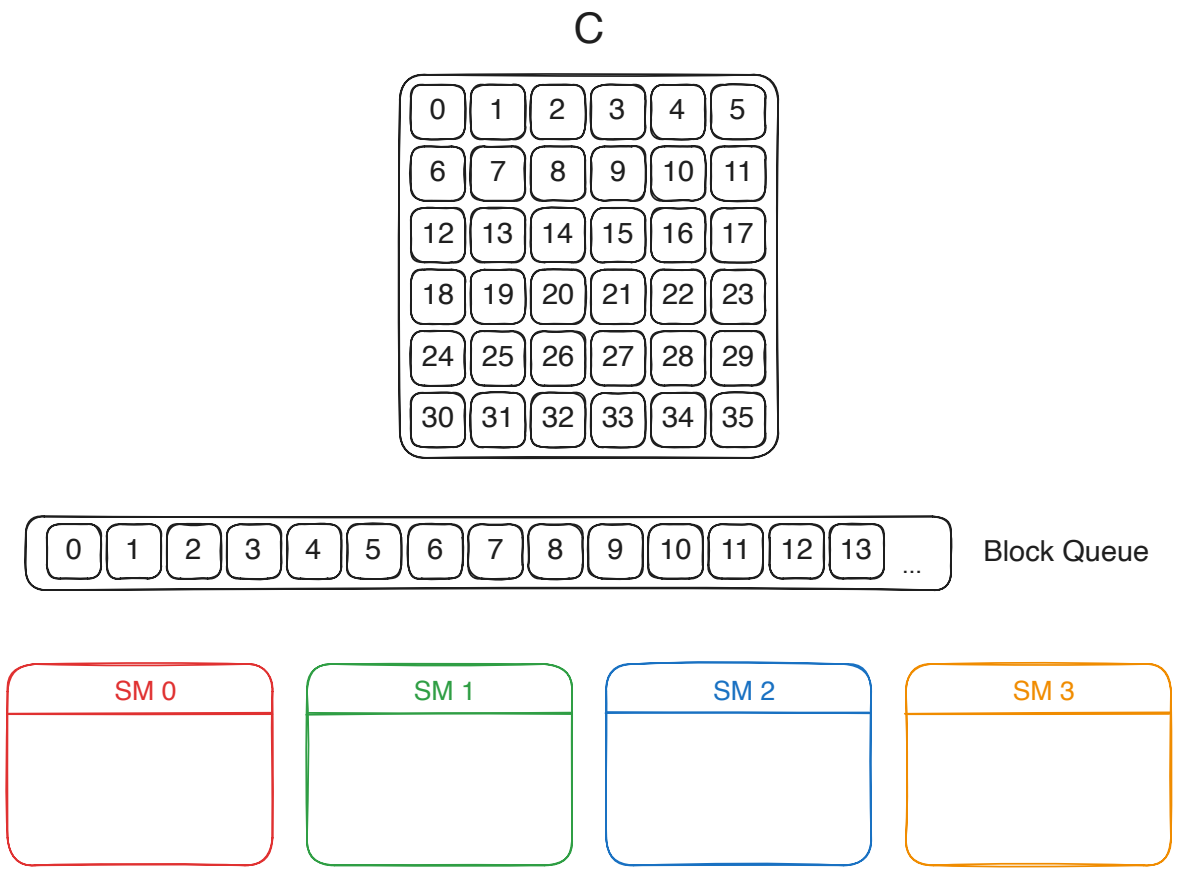
Assignment of Blocks to SMs

- There are no guarantees on the order in which blocks are assigned to SMs



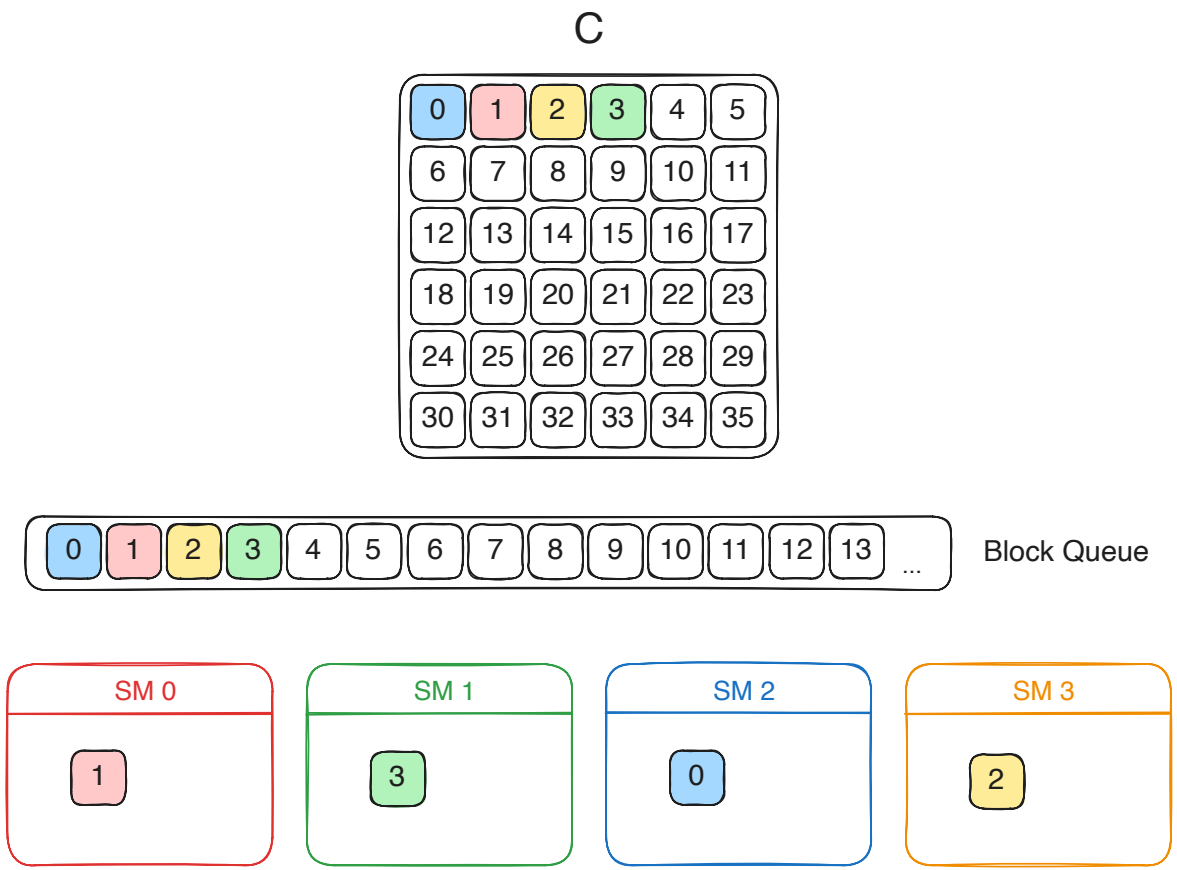
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



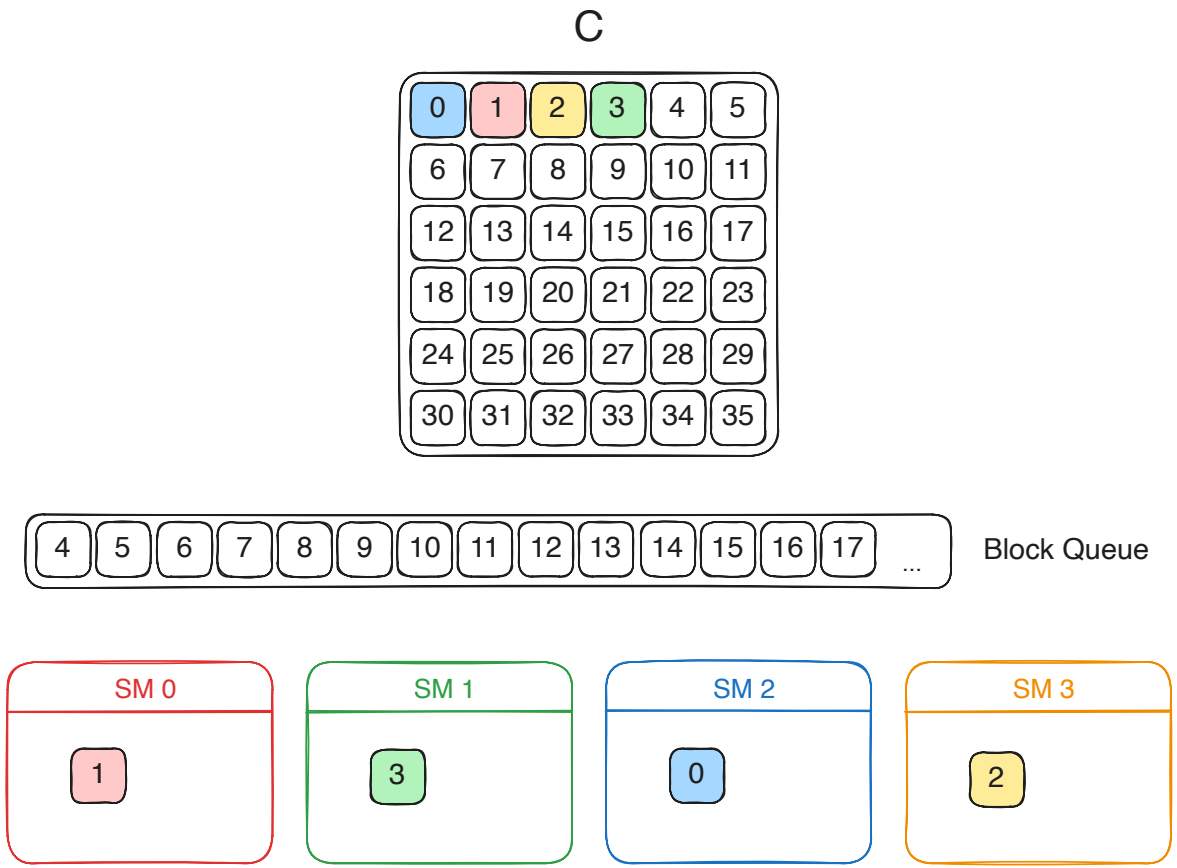
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



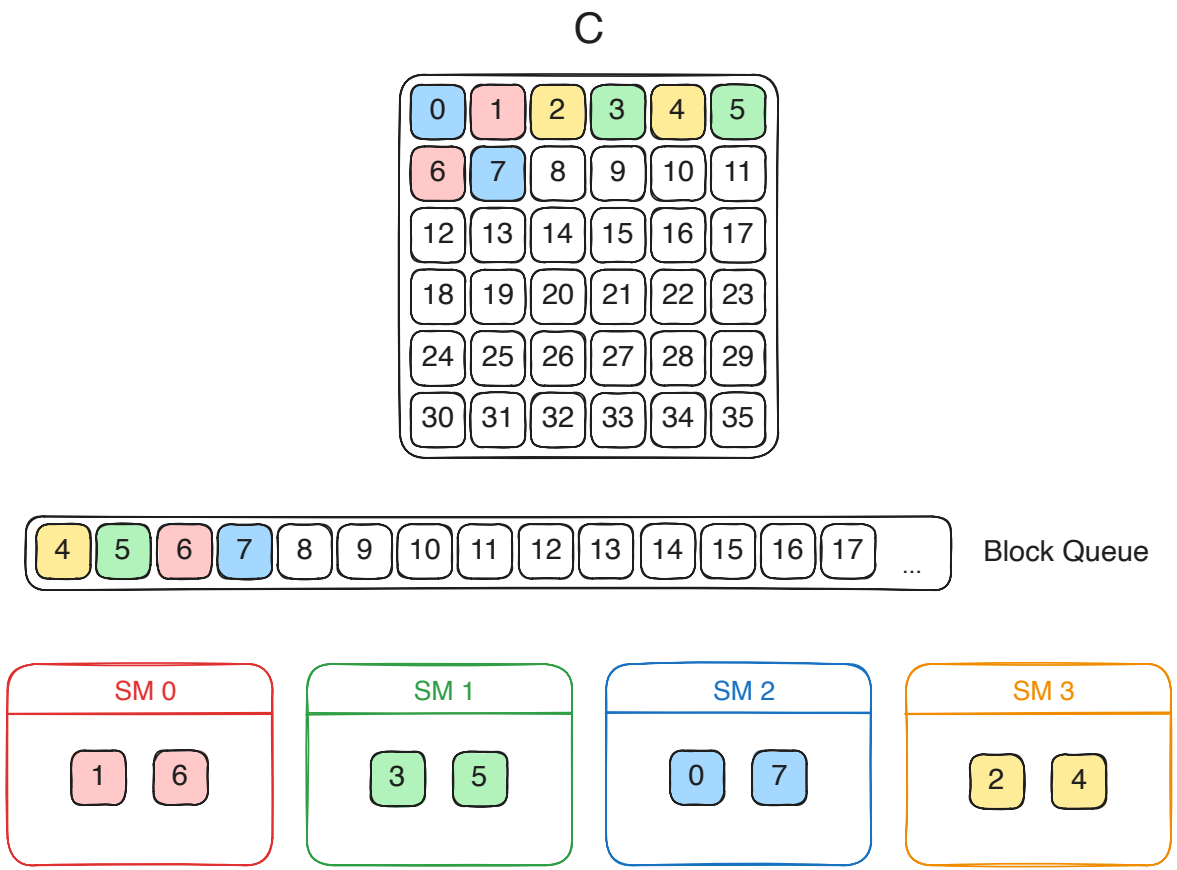
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



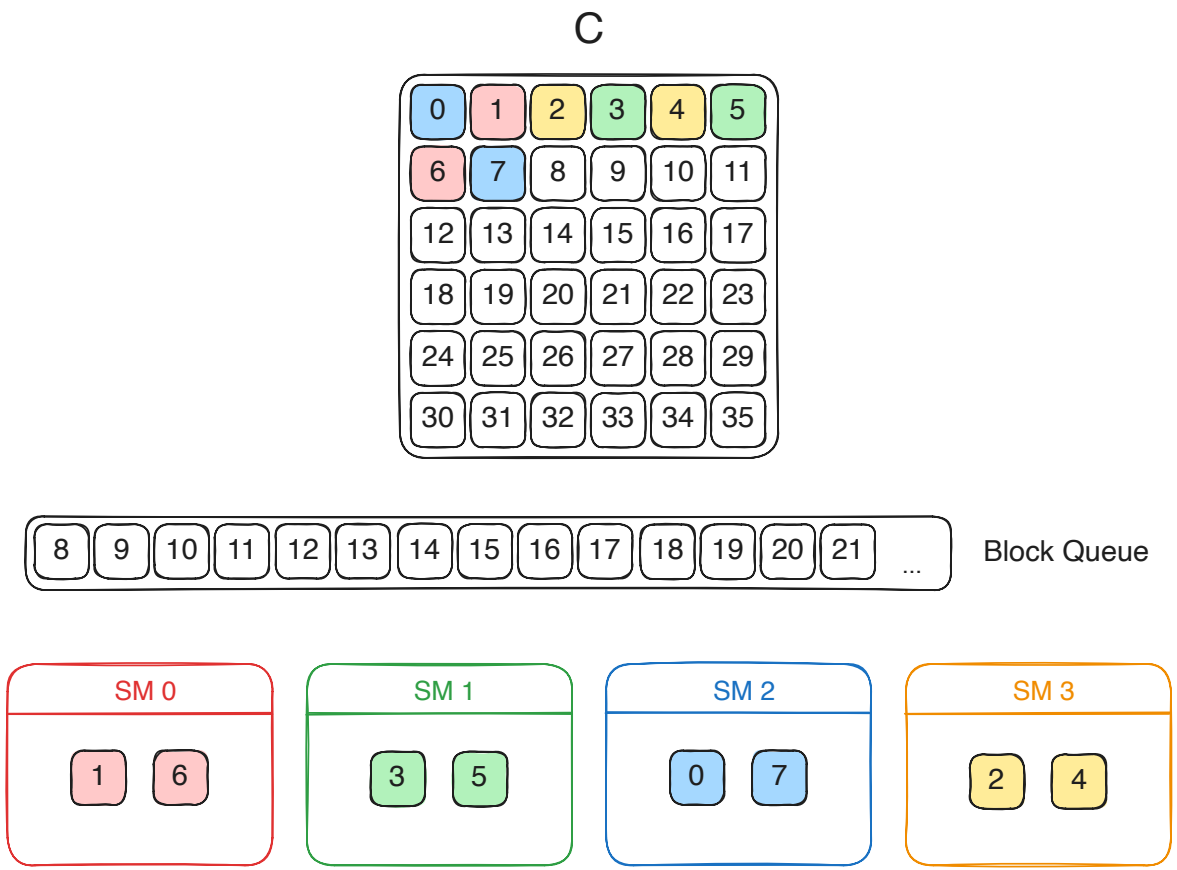
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



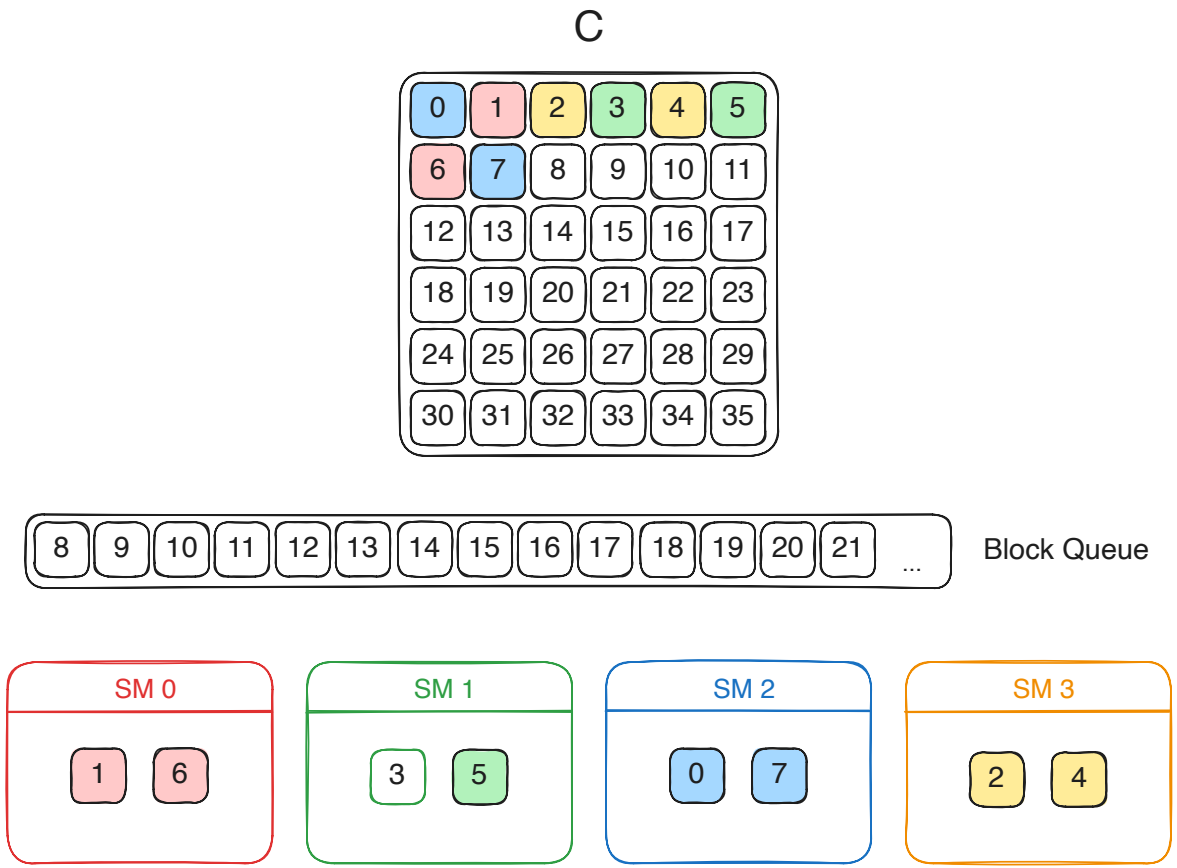
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



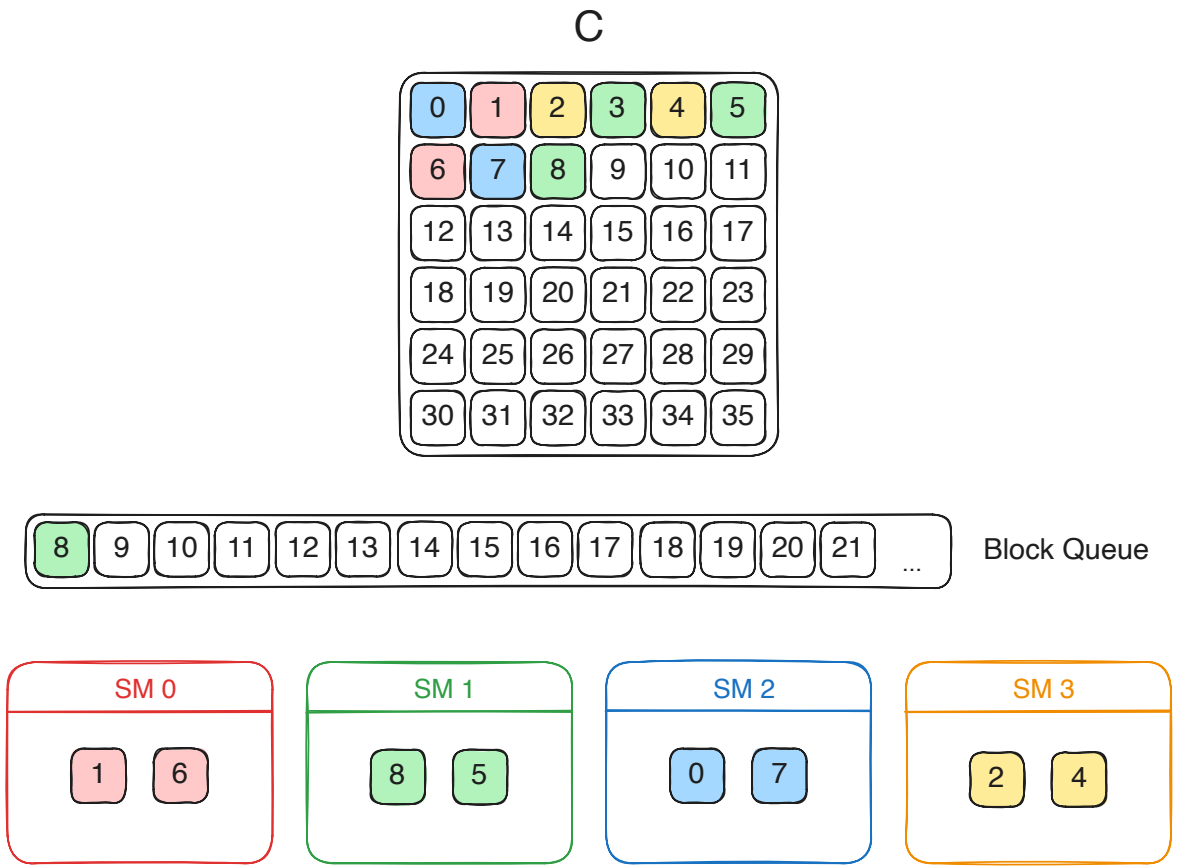
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



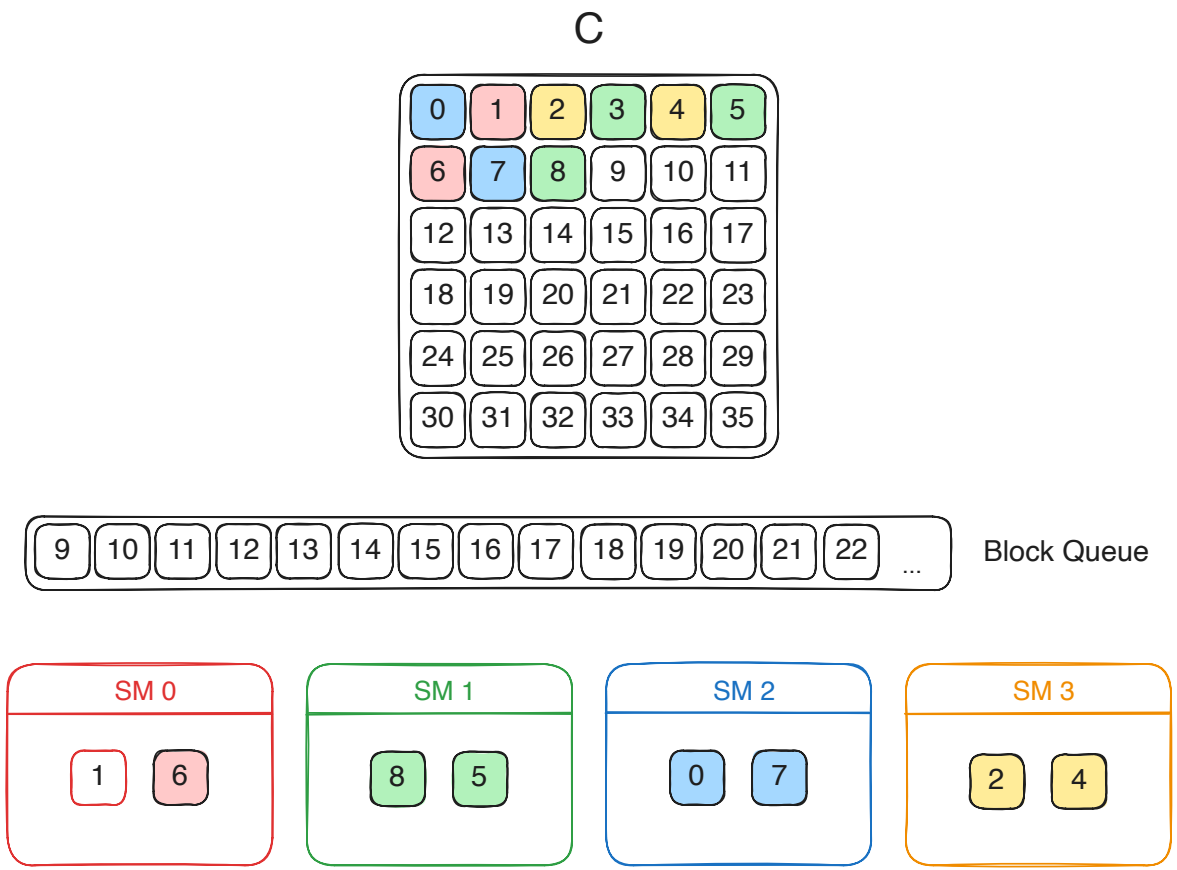
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



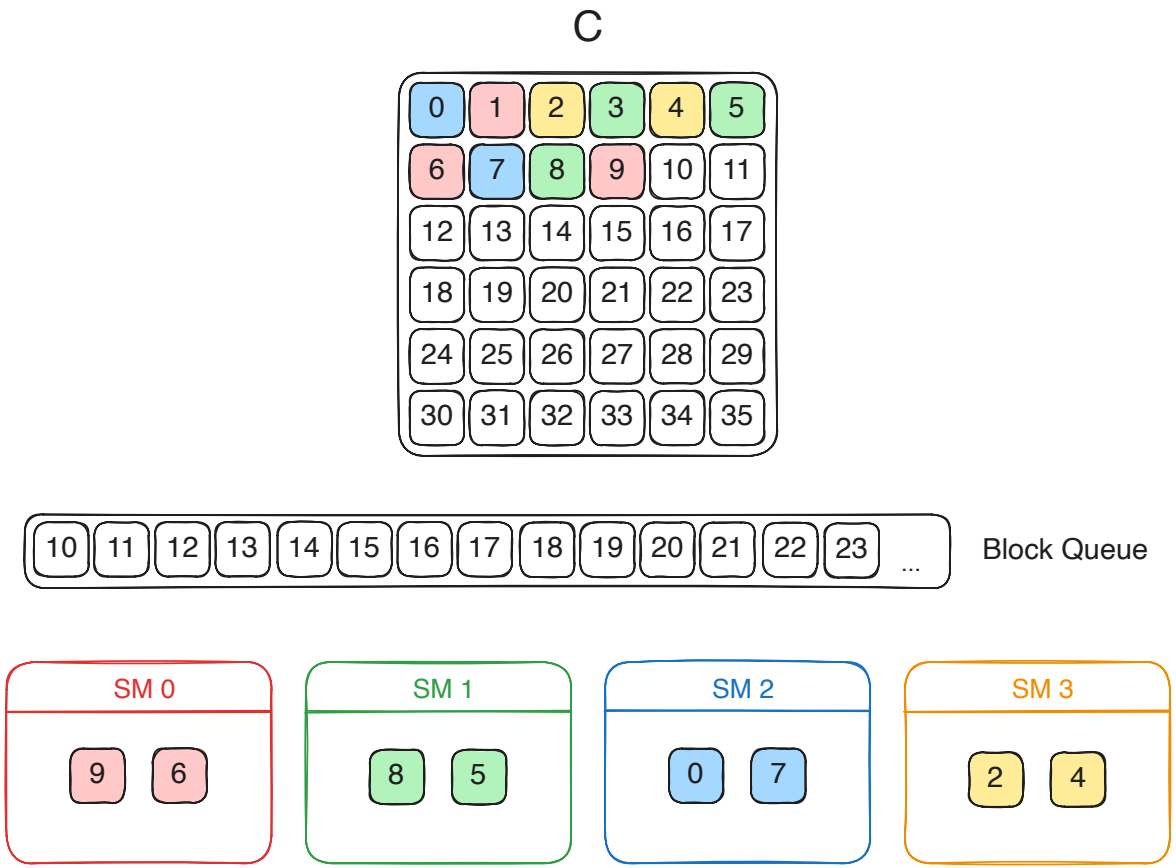
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



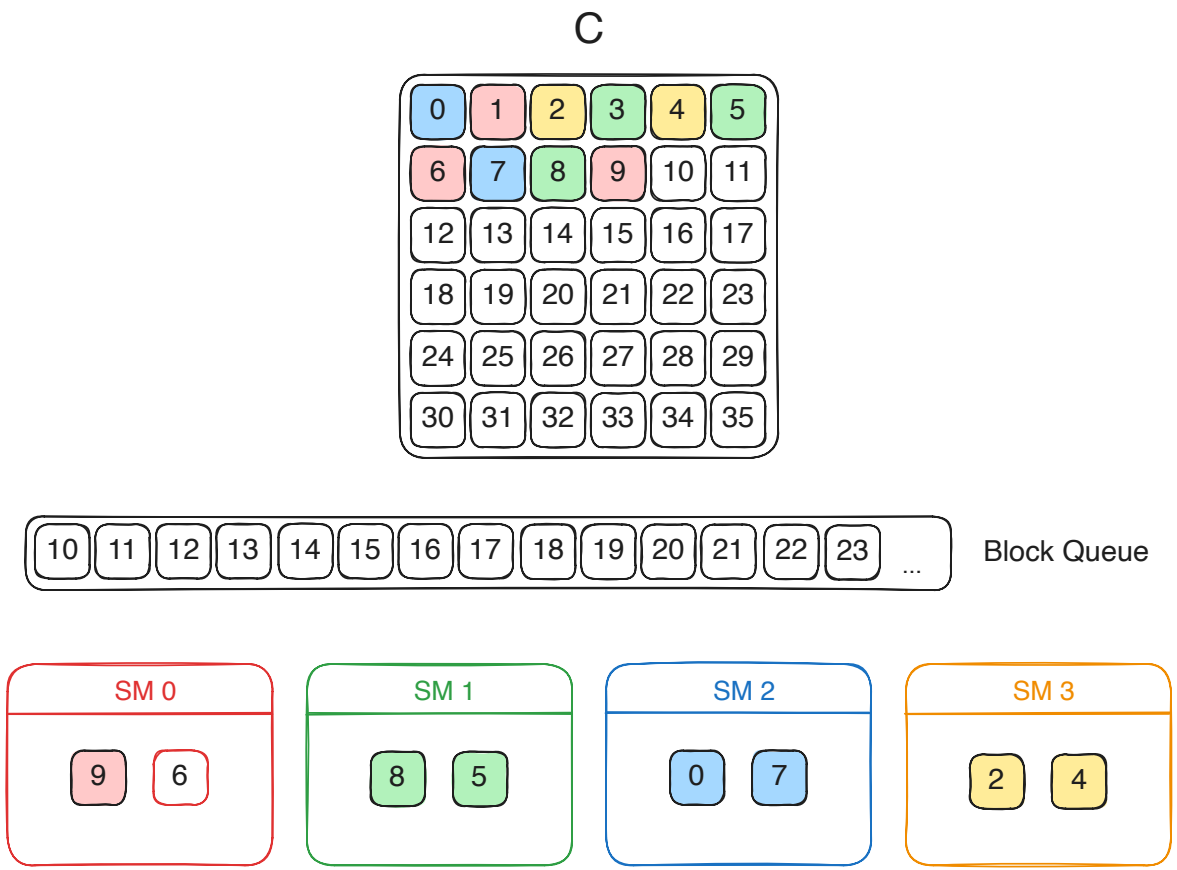
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID



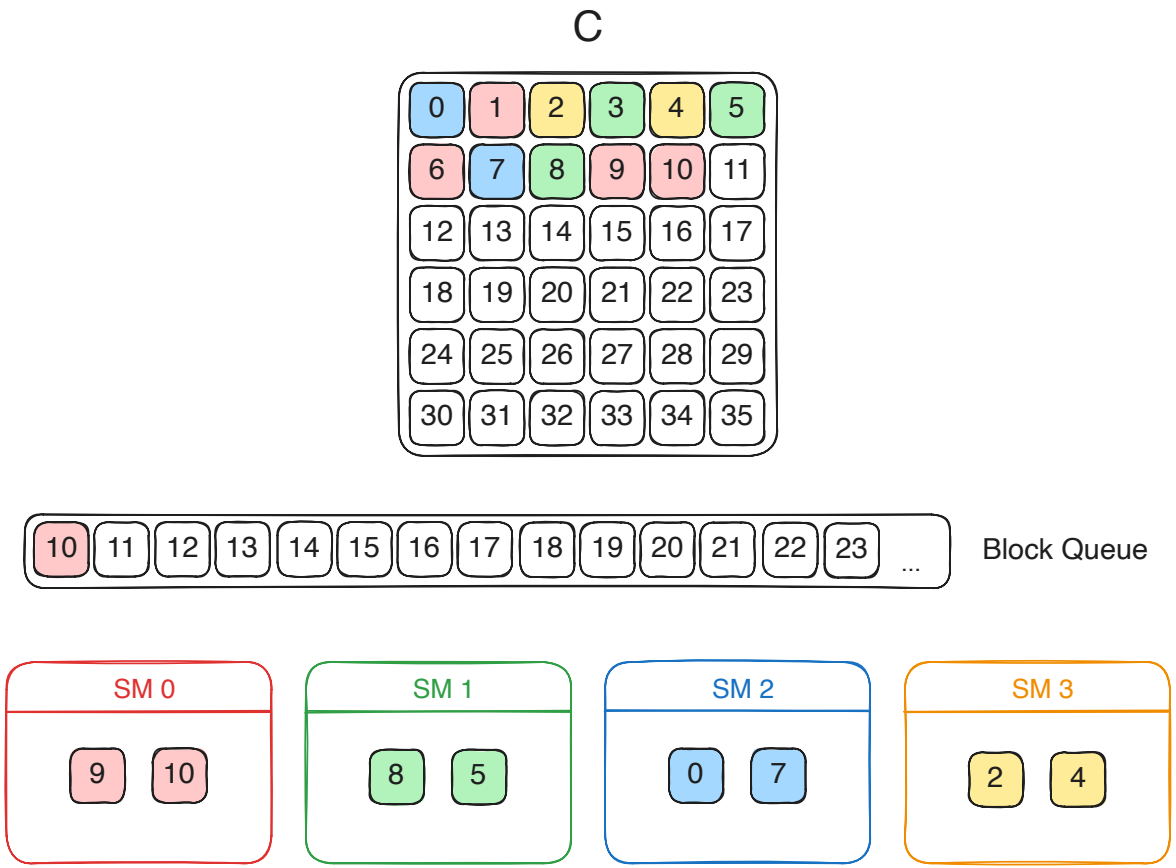
Scheduling of Blocks to SMs

- In practice, the hardware scheduler assigns blocks ordered by their block ID

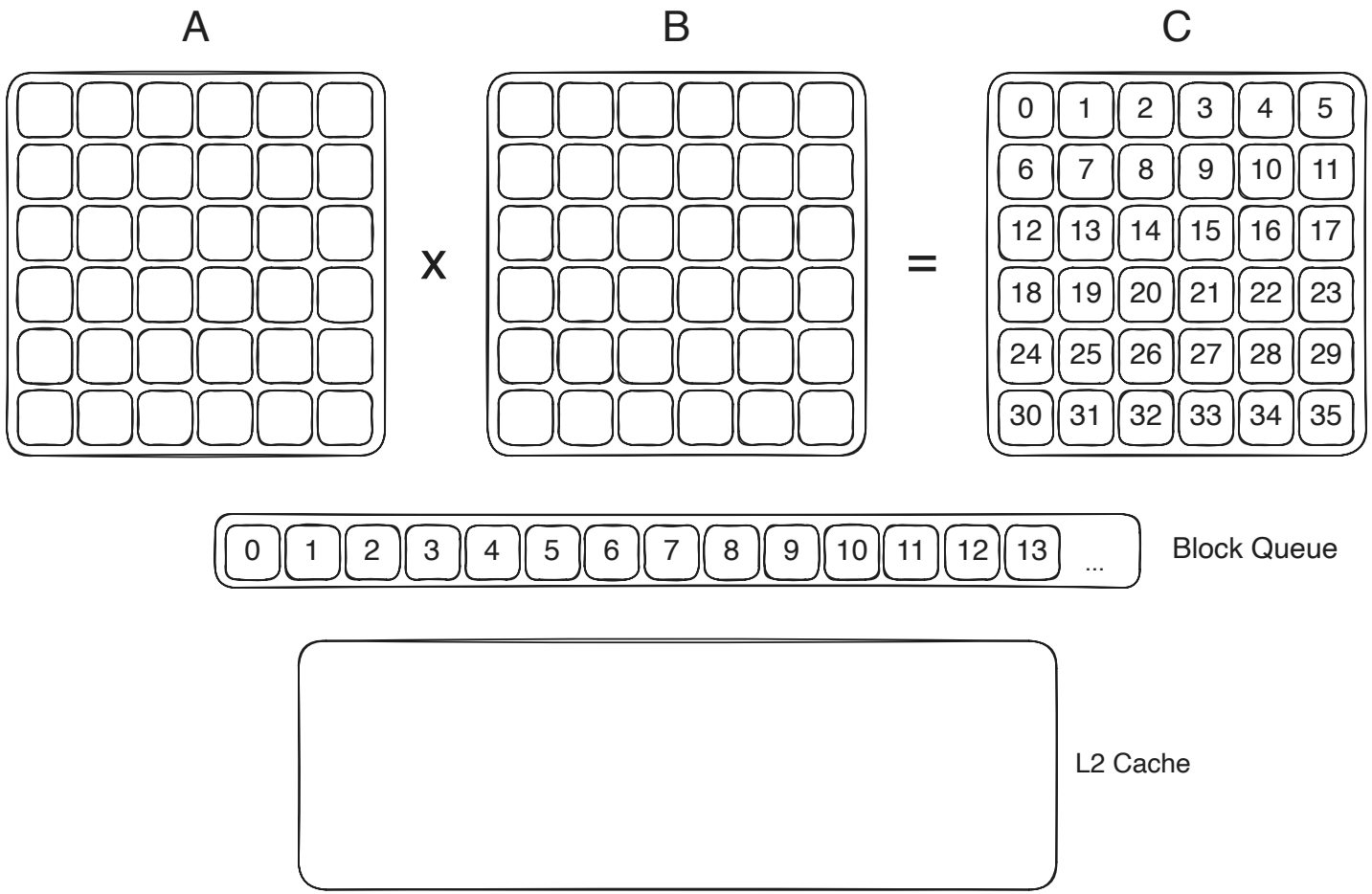


Scheduling of Blocks to SMs

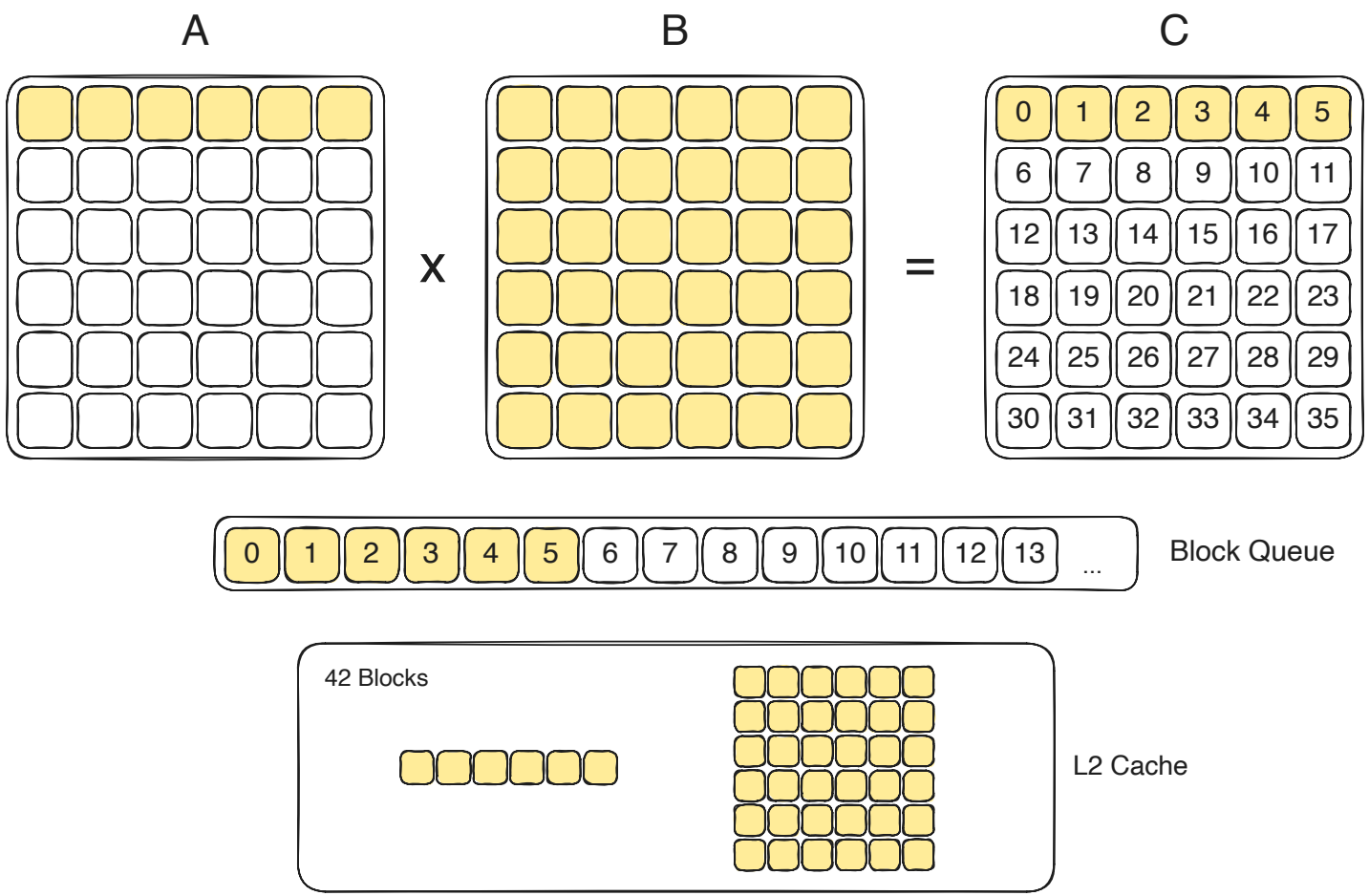
- In practice, the hardware scheduler assigns blocks ordered by their block ID



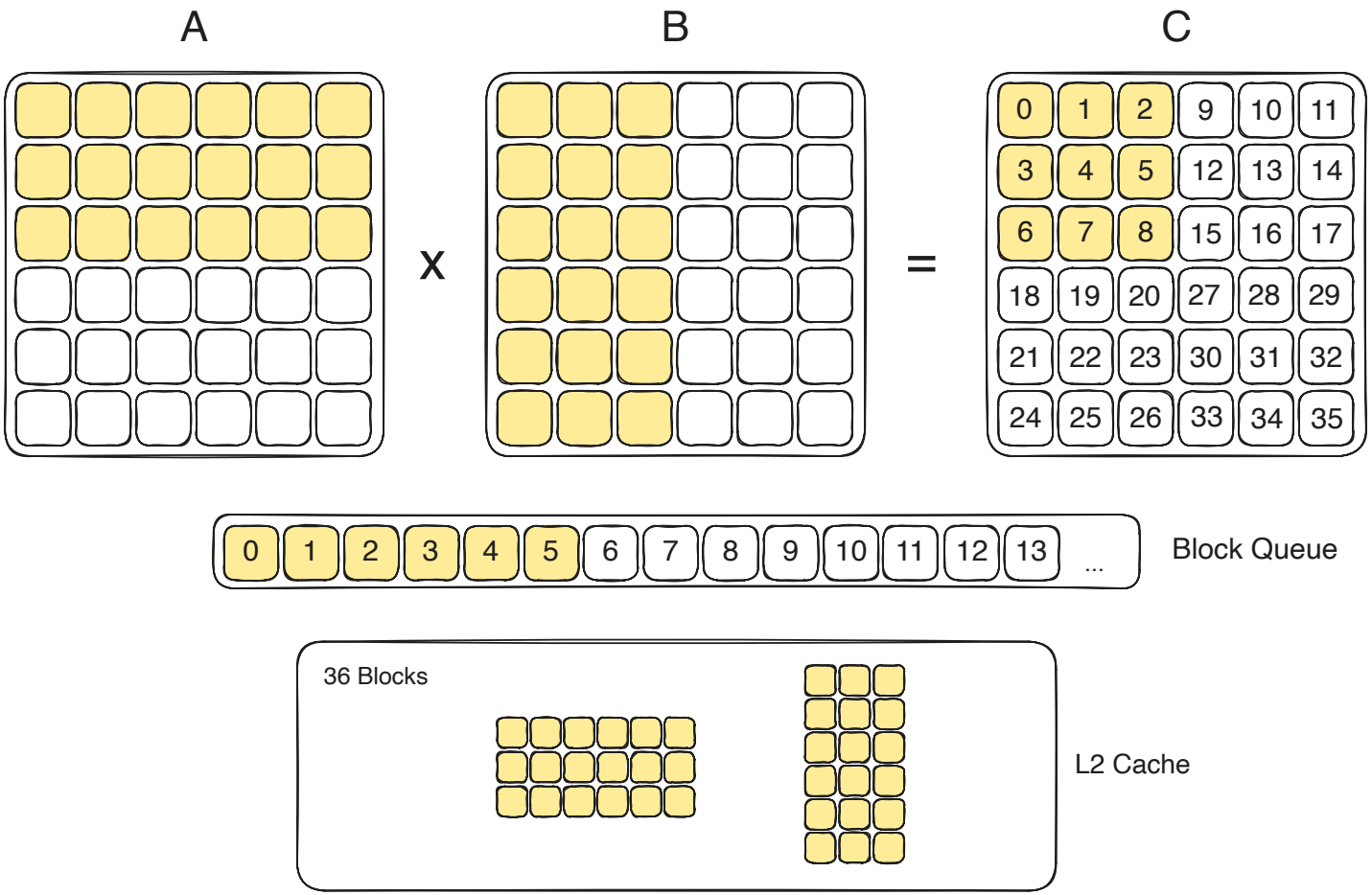
Effects of Block Scheduling on Cache Reuse



Effects of Block Scheduling on Cache Reuse



Thread Block Swizzling



Summary: Thread Block Assignment

- **In theory**, blocks can be scheduled in any order
- **In practice**, there is a strong bias towards scheduling blocks in order of their block IDs
- Naïve row-major block ID assignment can lead to poor L2 cache reuse
- **Thread block swizzling** can improve cache reuse and performance
  - Used to maximize **FLOPs / Bytes fetched from HBM**